Name: _____
Surname: _____
Date: _____

Debugging (15 points each)

Instructions

The functions below contain one or more errors. You have to find these errors, explain why they happen, and suggest a fix to them.

Questions

1. This code aims to count the number of unique words in a string, ignoring case sensitivity. However, it doesn't produce the correct count. Find the error and correct it.

```python
def count_unique_words(text):
    words = text.lower().split(" ")
    unique_words = set()
    for word in words:
        unique_words.add(word)
    return len(words)

sentence = "Hello world hello"
print(count_unique_words(sentence))  # Output: 3, Expected: 2
```

Answer:

The function builds the set `unique_words` correctly, as a set discards repeated elements. However, the last line returns `len(words)`, which is the length of the list with **all** the words, including the repeated ones. This is why the output is 3. *(+7 points)*

We need to return the length of the set instead, `return len(unique_words)`. In this example, the set is {`"hello", "world"`} and the function returns 2, as expected. *(+8 points)*

(Optional, fixed code)

```python
def count_unique_words(text):
    words = text.lower().split(" ")
    unique_words = set()
    for word in words:
        unique_words.add(word)
    return len(unique_words)
```

2. The following function is intended to find and return the maximum even number from a given list of integers. However, it sometimes returns incorrect results. Identify the issue(s) and propose a fix.

```python
def max_even_number(numbers: list[int]) -> int:
    max_even = None
    for num in numbers:
```

```
        if num % 2 == 0 and (max_even is None or num > max_even):
            max_even = num
        if max_even is None:
            return "No even number found"
        else:
            return max_even

nums = [1, 2, 3, 4]
print(max_even_number(nums))  # Prints "No even number found"

nums = [2, 4, 6]
print(max_even_number(nums))  # Prints 2
```

Answer:

The problem is the indentation of the last `if-else`. It is inside the for loop, so the function returns at the end of the *first* iteration, without looking at the rest of the numbers. For `[1, 2, 3, 4]`, the first number (1) is odd, so `max_even` is still `None` and the function returns the message right away. For `[2, 4, 6]`, it returns the first even number (2) instead of the maximum. *(+7 points)*

We fix it by moving the `if-else` out of the loop (reducing its indentation), so that it runs once, after all the numbers have been checked. *(+8 points)*

Comment: The type hint says that the function returns an `int`, but it returns a string when there are no even numbers. It would be better to return `None` in that case. This is not required here.

(Optional, fixed code)

```
def max_even_number(numbers: list[int]) -> int:
    max_even = None
    for num in numbers:
        if num % 2 == 0 and (max_even is None or num > max_even):
            max_even = num
    if max_even is None:
        return "No even number found"
    else:
        return max_even
```

3. The following code should join three arrays together to form the first nine letters of the abc. When you run it, the last line returns an error: "`AttributeError: "numpy.ndarray" object has no attribute "concatenate"`". Explain the issue(s) and fix it.

```
1 import numpy as np
2
3 nm = np.array(["a", "b", "c"])
4 nn = np.array(["d", "e", "f"])
5 np = np.array(["g", "h", "i"])
6
7 nq = np.concatenate([nm, nn, np])
```

Answer:

The error message is misleading, as the line that fails (the last one) is correct. The real problem is line 6: `np = np.array(["g", "h", "i"])`. The name `np` is the alias of the NumPy module, but this line assigns a new array to that same name, so `np` stops being the module and becomes an array. This is called shadowing. *(+8 points)*

Then, in the last line, Python looks for `concatenate` inside an array (which is now what `np` is), and an array has no such attribute. We fix it by giving the third array a different name, which does not overwrite `np`, for instance `nr`, and using that name inside `np.concatenate()`. *(+7 points)*

(Optional, fixed code)

```
1 import numpy as np
2
3 nm = np.array(["a", "b", "c"])
4 nn = np.array(["d", "e", "f"])
5 nr = np.array(["g", "h", "i"])
6
7 nq = np.concatenate([nm, nn, nr])
```

4. The function below should extract the antidiagonal of any square matrix. The code shows abnormal behaviour: it returns a result when a matrix is not square, and fails when the matrix is square. Explain the issue(s) and correct it.

```
1  import numpy as np
2
3  def antidiagonal(x: np.array) -> np.array:
4      if x.ndim != 2:
5          print("Array is not 2D")
6      elif x.shape[0] != x.shape[1]:
7          print("Matrix is not squared")
8
9      result = []
10     size = x.shape[0]
11
12     for i in range(size):
13         j = size - i
14         result.append(x[i, j])
15
16     return np.array(result)
```

```
17
18 b = np.array(
19     [[1, 2, 3],
20      [4, 5, 6]]
21 )
22 print(antidiagonal(b))
23 # Expected: Matrix is not squared
24 # Output (two lines):
25 # Matrix is not squared
26 # [3 5]
27
28 a = np.array(
29     [[1, 2],
30      [3, 4]]
31 )
32 print(antidiagonal(a))
33 # Expected: [2, 3]
34 # Output:
35 # IndexError: index 2 is out of bounds for axis 1 with size 2
```

Answer:

The first problem is that the checks in lines 4 to 7 only print a message, and the function continues anyway. For a non-square matrix, the code goes on to compute (wrong) values and returns them. After printing each message, we need to stop the function with a `return` (for instance `return None`). Another valid fix is to place the rest of the code inside an `else` clause. *(+7 points)*

The second problem is the column index in line 14. The columns of the antidiagonal go from the last one to the first one, and the last column has index `size - 1`, not `size`. For `i = 0`, `j = size - 0 = size` is out of bounds, which raises the `IndexError` for the 2x2 matrix. We need `j = size - i - 1`. *(+8 points)*

Comment: With the `return None`, `print(antidiagonal(b))` would still show `None` after the message, because the function has no result to give.

(Optional, fixed code)

```
1  import numpy as np
2
3  def antidiagonal(x: np.array) -> np.array:
4      if x.ndim != 2:
5          print("Array is not 2D")
6          return None
7      elif x.shape[0] != x.shape[1]:
8          print("Matrix is not squared")
9          return None
10
11     result = []
12     size = x.shape[0]
13
14     for i in range(size):
15         j = size - i - 1
16         result.append(x[i, j])
17
18     return np.array(result)
```

Pseudo-Code

Instructions

In the following exercises, you will be asked to write Python functions by hand. Please follow these guidelines:

- Any build-in function you need is in the cheat-sheet provided.
- Clearly indicate each level of indentation using a right arrow (⇒).
- Ensure your code works correctly for any possible example.
- If your code has errors, it will be evaluated based on the provided docstring, comments, and explanations.

Full points will be awarded for code that performs as expected under all conditions.

5. Removing Characters (20 points)

Create the `remove_first_char()` function to take a string and return a new version. This version should remove all occurrences of the first character, except the first one itself. For example, "sun shines" becomes "sun hine" (all "s" except the first were removed). Note that the function treats lowercase and uppercase as distinct characters.

Answer:

The function receives a string and returns a new string. We need two pieces: the first character, which always stays, and the rest of the string, from which we remove every occurrence of that first character. *(+5 points)*

We store the first character with `first = s[0]`. The rest of the string is `s[1:]`. To remove all occurrences of `first` from the rest, we use `s[1:].replace(first, "")`, which returns a new string with the character deleted. Because `replace()` is case sensitive, uppercase and lowercase characters are treated as different, as the problem requires. *(+10 points)*

Finally, we join the first character and the cleaned rest with `+`, and return the result. *(+5 points)*

Comment: If we allowed an empty string, `s[0]` would fail. It is a good practice to return the empty string in that case. A solution with a loop that builds the new string character by character, skipping the ones equal to the first (except the first position), is also valid.

(Optional, code)

```
def remove_first_char(s: str) -> str:
    if len(s) == 0:
        return s
    first = s[0]
    rest = s[1:].replace(first, "")
    return first + rest
```

6. Circular Motion Coordinates (20 points)

Your task is to simulate the circular motion of an object by computing its coordinates. For a circle with radius R and an object moving along its circumference, you need to determine the object's x and y positions corresponding to a list of angles α (in degrees).

The coordinates are determined by the following formulas:

$$x = R \cdot \cos(\alpha)$$
$$y = R \cdot \sin(\alpha)$$

Part 1 (10 points): Implement the function `circular_motion(radius, angles)`:

- `radius`: A scalar value denoting the radius of the circle, R.
- `angles`: A list of angles in **degrees**, α.

The function should compute and return two NumPy arrays: `x_coords` and `y_coords`. These arrays represent the x and y coordinates on the circle for each angle in `angles`. Round this values to two decimals.

Part 2 (10 points): Implement a function `verify_coordinates(radius, x_coords, y_coords)` to verify the accuracy of your circular motion function. For each coordinate pair (x, y) in `x_coords` and `y_coords`, check if the equation $\sqrt{x^2 + y^2}$ equals the radius R within a reasonable tolerance (e.g., 0.01). This validation ensures that each calculated point lies on the circle. The function should return "True" if all points are on the circle and "False" otherwise.

Answer:

Part 1. The formulas need the angle in radians, but the input is in degrees. The function `np.radians()` converts it, and it works with a whole list at once (or array). Because we are going to use arrays, we first convert the list with `np.array(angles)`. *(+3 points)*

Then, thanks to vectorization, we compute all the coordinates without a loop: `x_coords = radius * np.cos(rad)` and `y_coords = radius * np.sin(rad)`. *(+4 points)*

The problem asks for two decimals, so we round both arrays with `np.round(array, 2)` (or `.round(2)`), and return the two arrays. *(+3 points)*

Part 2. The function receives the radius and the two arrays. For each point, we compute the distance to the origin, `np.sqrt(x_coords**2 + y_coords**2)`, using the arrays directly. *(+3 points)*

Next, we compare each distance with the radius, allowing a tolerance: `np.abs(distance - radius) <= 0.01`. We cannot use `==` because the coordinates were rounded, so the distances are not exactly equal to the radius. *(+4 points)*

This gives a boolean array. The function returns `True` only if all of its values are true, which we check with `np.all()`, and `False` otherwise. *(+3 points)*

(Optional, code)

```python
import numpy as np

def circular_motion(radius, angles):
    # Degrees to radians
    rad = np.radians(np.array(angles))
    x_coords = np.round(radius * np.cos(rad), 2)
    y_coords = np.round(radius * np.sin(rad), 2)
    return x_coords, y_coords

def verify_coordinates(radius, x_coords, y_coords):
    # Distance from each point to the origin
    distance = np.sqrt(x_coords**2 + y_coords**2)
    # True only if every point is close enough to the circle
    return bool(np.all(np.abs(distance - radius) <= 0.01))
```

Cheatsheet

Given a number named `x`:

- `x % y` returns the remainder of the division $\frac{x}{y}$.
- `x // y` returns the quotient of the division $\frac{x}{y}$.
- `abs(x)` returns the absolute value of `x`.

Given a string named `s`:

- `int(s)` returns an integer. Fails if `s` has invalid characters.
- `float(s)` returns a floating number. Fails if `s` has invalid characters.
- `s.lower()` returns a copy of the string, all in lowercase.
- `s.split(" ")` split a string into a list where each word is a list item.
- `s.count(<pattern>)` returns the number of non-overlapping occurrences of `<pattern>`.
- `s.find(<pattern>, <start>)` finds the first occurrence of the specified `<pattern>`, starting from index `<start>`. Returns `-1` if the pattern is not found.
- `s.replace(<pattern>, <new>)` returns a new string where all instances of `<pattern>` have been replaced with `<new>`.

Given a list named `ls`:

- `ls.append(element)` add element to that list.

Given a list or tuple named `ls`:

- `ls.index(element)` returns the index of element in that list.
- `len(ls)` returns the number of elements in that list.
- `sum(ls)` returns the sum of the elements of that list.
- `sorted(ls)` returns a sorted version of that list.
- `tuple(ls)` / `set(ls)` returns a tuple / set using the elements of that list.
- `", ".join(ls)` returns a string with the elements of the list separated by commas.

Given a set named `a`:

- `a.add(x)` adds the element `x` to the set `a`.
- `a.union(b)` or (`a | b`) returns the union of set `a` with another set `b`.
- `a.intersection(b)` or (`a & b`) returns the intersection of set `a` with another set `b`.
- `a.difference(b)` or (`a - b`) returns the difference of set `a` with another set `b`.

Given a dictionary named `d`:

- `list(d.keys())` returns a list with the keys of `d`.
- `list(d.values())` returns a list with the values of `d`.
- `list(d.items())` returns a list with the tuples (key, value) of `d`.

Some useful build-in functions:

- `type(x)` returns the type of `x` (int, float, list...).
- `range(start, stop)` generates numbers from `start` until `stop`, not including `stop`.
- `input(msg)` is used to take user input, in form of a string.

NumPy Cheatsheet

Importing NumPy:

- `import numpy as np`
- `rng = np.random.default_rng()`

Creating arrays:

- `np.zeros((m, n))` creates an $m \times n$ array filled with zeros.
- `np.ones((m, n))` creates an $m \times n$ array filled with ones.
- `np.arange(start, stop, step)` creates values from `start` to `stop` (exclusive) with a given `step`.
- `np.linspace(start, stop, num)` creates `num` evenly spaced values between `start` and `stop`.

Given a NumPy array `arr`:

- `arr.ndim` returns the number of array dimensions, as a number.
- `arr.shape` returns the shape of an array, as a tuple.

Array operations:

- `np.dot(a, b)` computes the dot product.
- `np.matmul(a, b)` or `a @ b` performs matrix multiplication.
- `np.transpose(a)` or `a.T` transposes the array.

Array manipulation:

- `a.reshape((m, n))` changes the shape of an array.
- `a.flatten()` flattens a multidimensional array into 1D.
- `np.concatenate((a, b), axis=0)` joins arrays along an axis.

- `np.stack(<sequence>)` joins a sequence of arrays along a new axis.
- `np.vstack((a, b))`, `np.hstack((a, b))` vertical/horizontal stacking.
- `np.split(a, n)` splits an array into `n` parts.

Statistics and descriptive analysis:

- `np.mean(a)` mean of all elements.
- `np.median(a)` median of all elements.
- `np.std(a)` standard deviation.
- `np.var(a)` variance.
- `np.min(a)`, `np.max(a)` minimum and maximum.
- `np.argmin(a)`, `np.argmax(a)` indices of minimum and maximum.
- `np.corrcoef(a, b)` correlation coefficient matrix.
- `np.cov(a, b)` covariance matrix.

Linear algebra:

- `np.linalg.inv(a)` inverse of matrix `a`.
- `np.linalg.det(a)` determinant of `a`.
- `np.linalg.eig(a)` eigenvalues and eigenvectors.
- `np.linalg.norm(a)` vector or matrix norm.
- `np.linalg.solve(a, b)` solves $Ax = b$.

Logical operations and conditions:

- `np.where(cond, x, y)` selects elements from `x` or `y` based on condition.
- `np.all(a)`, `np.any(a)` check if all/any elements are True.
- `np.unique(a)` returns sorted unique elements.

Random number generation (using `rng`):

- `rng.random(size)` uniform distribution in $[0, 1)$.
- `rng.integers(low, high, size)` random integers in $[low, high)$.
- `rng.normal(mean, std, size)` normal (Gaussian) distribution.
- `rng.uniform(low, high, size)` uniform distribution in $[low, high)$.
- `rng.choice(a, size)` random sample from array `a`.

Random number generation (legacy API):

- `np.random.randint(low, high, size)` returns random integers from low (inclusive) to high (exclusive).
- `np.random.uniform(low, high, size)` samples from a uniform distribution.
- `np.random.seed(seed)` fixes the seed. After a seed reset, the same set of numbers will appear every time.

Math and trigonometry:

- `np.power(arr1, arr2)` returns the first array elements raised to powers from second array, element-wise.
- `np.sin(arr)` returns the trigonometric sine, assuming angle in radians.
- `np.cos(arr)` returns the trigonometric cosine, assuming angle in radians.
- `np.radians(arr)` converts angles from degrees to radians.
- `np.degrees(arr)` converts angles from radians to degrees.